

Ergebnisbericht: Schnappschuss eines konsistenten globalen Zustands

Verteilte Systeme Übungsblatt 2

In diesem Übungsblatt ging es darum, während des laufenden Betriebs einer verteilten Anwendung einen konsistenten globalen Zustand aufzuzeichnen. In einem verteilten System gibt es keine gemeinsame Uhr und keinen Ort, an dem man den Gesamtzustand einfach ablesen könnte. Jeder Prozess kennt nur seinen eigenen lokalen Zustand und dazu kommen Nachrichten, die gerade unterwegs sind und in diesem Moment zu keinem Prozess gehören. Ein Schnappschuss muss deshalb beides erfassen: die lokalen Zustände aller Prozesse und die Kanalzustände, also alle Nachrichten, die zum Zeitpunkt des Schnitts schon abgeschickt, aber noch nicht empfangen waren.

Als Testanwendung habe ich das im Aufgabenblatt beschriebene „Bankhaus“ im Simulator sim4da umgesetzt: n Prozesse mit je einem Konto überweisen sich ständig gegenseitig Geld. Weil dabei kein Geld verloren geht oder dazukommt, gilt jederzeit, dass die Summe aller Kontostände plus aller gerade unterwegs befindlichen Überweisungen konstant gleich der Startsumme S ist. Das ist praktisch, weil sich damit sofort prüfen lässt, ob ein Schnappschuss konsistent ist: Kommt genau S heraus, stimmt der Snapshot, während alles andere ein inkonsistenter Zustand ist.

Aufgabe 1: das „Bankhaus“

Jeder der n Prozesse (standardmäßig $n = 5$) startet mit einem Saldo von 1000, die Startsumme ist also $S = 5000$. Alle Prozesse sind vollständig vernetzt. Ein Prozess wartet wiederholt eine kurze zufällige Zeit (50–300 ms), zieht dann einen zufälligen Betrag b mit $0 < b \leq \text{Saldo}$ von seinem eigenen Konto ab und schickt eine Nachricht *Transfer(b)* an einen zufällig gewählten anderen Prozess, der den Betrag beim Empfang gutschreibt. Weil *receive()* in sim4da blockiert, schickt sich jeder Prozess selbst eine kleine Tick-Nachricht, die die nächste eigene Überweisung anstößt. So laufen Senden und Empfangen verschränkt in einer einzigen Schleife und es wird keinerlei Synchronisation auf dem Kontostand benötigt.

Wie im Aufgabenblatt gefordert, habe ich das Standardverhalten des Simulators an zwei Stellen „verschärft“. Erstens gibt es jetzt eine Nachrichtenlatenz: Die Zustellung jeder Überweisung wird um eine zufällige Übertragungszeit von 100 bis 1000 ms verzögert (über einen kurzen Hilfs-Thread, der schläft und dann sendet). Wichtig ist, dass das Konto trotzdem sofort beim Absenden reduziert wird, wodurch das Geld während der Latenz wirklich unterwegs ist und weder zum Sender- noch zum Empfängerkonto gehört. Zweitens ist die FIFO-Zustellung abgeschaltet: Über die im Aufgabenblatt angegebene Verteilungsfunktion nimmt jeder Empfänger eine zufällige Nachricht aus seiner Warteschlange, Nachrichten können sich also gegenseitig überholen.

Wie viel Geld tatsächlich unterwegs ist, sieht man schon an der Anwendung selbst. In meinem Beispiellauf summierten sich die fünf Endsalden beim Abschalten nur auf 1202 von 5000, während die restlichen 3798 in Überweisungen steckten, die noch nicht zugestellt waren. Spätestens da war klar, dass es nicht reicht, für einen Schnappschuss nur die Konten abzufragen.

Aufgabe 2: Snapshot mit Chandy-Lamport

Von den beiden erlaubten Varianten habe ich Chandy-Lamport gewählt, also das dezentrale Verfahren. Es braucht keinen eigenen Koordinator-Prozess, sondern ein beliebiger Prozess (bei mir Prozess 0) initiiert den Schnappschuss selbst. Der Initiator sichert seinen lokalen Zustand und schickt über jeden ausgehenden Kanal eine Marker-Nachricht. Wer zum ersten Mal einen Marker empfängt, sichert ebenfalls seinen Zustand und beginnt, auf allen übrigen Eingangskanälen die ankommenden Nachrichten aufzuzeichnen. Das sind genau die Nachrichten, die beim Schnitt unterwegs waren. Ein erneuter Marker auf einem Kanal beendet dessen Aufzeichnung. Am Ende werden die Teilzustände beim Initiator eingesammelt.

Beim Implementieren bin ich schnell an den Punkt gekommen, an dem die Grundform nicht mehr funktioniert und zwar genau wegen der Vorgabe aus dem Aufgabenblatt, die FIFO-Zustellung abzuschalten. Durch Chandy-Lamport kann der Marker auf einem Kanal nicht überholt werden: Alles, was auf diesem Kanal vor dem Marker ankommt, wurde vor dem Schnitt abgeschickt, der Rest danach. Bei zufälliger Nachrichtenauswahl aus der Warteschlange stimmt diese Logik nicht mehr und es geht auf zwei Arten schief. Erstens kann eine alte Überweisung den Marker überholen lassen, also erst nach ihm ankommen. Dann hätte die Kanalaufzeichnung schon geendet und die Überweisung würde fehlen, wodurch Geld im Schnappschuss verschwunden wäre. Zweitens kann umgekehrt eine Überweisung, die der Absender erst nach seinem Schnappschuss losgeschickt hat, vor dem Marker eintreffen. Der Empfänger hätte seinen Zustand noch nicht gesichert und würde die Nachricht mitzählen, obwohl ihr Absenden hinter dem Schnitt liegt. Wir hätten somit eine Nachricht aus der „Zukunft“ und das Geld würde doppelt gezählt.

Gelöst habe ich das mit zwei Ergänzungen, die die FIFO-Annahme ersetzen. Die erste: Jede Überweisung bekommt ein Flag *pre*, das angibt, ob sie abgeschickt wurde, bevor der Absender seinen Zustand gesichert hat. Damit steckt die Information „vor oder nach dem Schnitt“ in der Nachricht selbst und hängt nicht mehr an der Reihenfolge, in der sie zugestellt wird. Trifft eine *post*-Nachricht bei einem Prozess ein, der noch keinen Marker gesehen hat, sichert er einfach zuerst seinen Zustand und verbucht sie danach. Die *post*-Nachricht wirkt dann praktisch wie ein vorgezogener Marker und das Problem mit der Nachricht aus der Zukunft wäre gelöst.

Die zweite Ergänzung betrifft das Ende der Kanalaufzeichnung. „Ein erneuter Marker beendet die Aufzeichnung“ funktioniert ohne FIFO nicht, weil der Marker selbst überholt worden sein kann. Stattdessen trägt bei mir jeder Marker die Anzahl der *pre*-Überweisungen, die der Absender auf diesem Kanal losgeschickt hat. Der Empfänger führt dazu drei Zähler pro Kanal: wie viele *pre*-Nachrichten er vor seinem eigenen Schnappschuss empfangen hat (die stecken schon im gesicherten Saldo), wie viele er danach aufgezeichnet hat (das ist der Kanalzustand) und wie viele laut Marker insgesamt kommen müssen. Ein Kanal ist fertig, sobald die Bilanz aufgeht. Das löst nebenbei auch die Terminierungsfrage, die in der Grundform gar nicht so offensichtlich ist: Sind alle Kanäle eines Prozesses fertig, schickt er seinen Teilzustand (gesicherten Saldo plus aufgezeichnete Kanalinhalte) an den Initiator und der gibt den vollständigen globalen Zustand aus, sobald alle n Teilzustände da sind.

Zwei kleinere Stolpersteine sind mir bei der Umsetzung noch begegnet. Zum einen hat `sim4da` keine expliziten Kanal-Objekte. Ein „Kanal“ ist bei mir einfach das Paar aus Absender und Empfänger und der Marker geht als normale Nachricht einzeln an jeden anderen Prozess. Er unterliegt damit derselben Umsortierung wie alles andere, was aber dank der Zähler egal ist. Zum anderen musste ich aufpassen, wann das *pre*-Flag gesetzt wird: Wegen der künstlichen Latenz wird eine Überweisung erst

deutlich nach der Abbuchung wirklich versendet. Logisch gilt sie aber ab der Abbuchung als unterwegs, also wird das Flag (und der Sendezähler) im Moment der Abbuchung festgelegt und nicht erst im Latenz-Thread. Mit dem Flag im falschen Moment wäre der Schnappschuss subtil falsch geworden.

Meine Umsetzung setzt voraus, dass keine Nachrichten verloren gehen und keine Prozesse abstürzen, was beides durch den Simulator garantiert wird. FIFO-Kanäle setze ich dagegen ausdrücklich nicht voraus, das übernehmen Flag und Zähler. Für die Experimente in Aufgabe 3 brauchte ich außerdem mehrere Schnappschüsse nacheinander. Dafür habe ich das *pre*-Flag zu einer Rundennummer verallgemeinert. Runde *r* heißt „*r*-mal Zustand gesichert“, jede Nachricht trägt die Rundennummer ihres Absenders und die Fallunterscheidung bleibt dieselbe: kleinere Runde als der Empfänger bedeutet Kanalzustand, größere bedeutet Nachricht aus der Zukunft.

Aufgabe 3: Konsistenz-Experimente

Zum Nachweis der Konsistenz habe ich Schnappschüsse ausgelöst, während die Überweisungen ununterbrochen weiterliefen und das Ergebnis war in sämtlichen Läufen dasselbe: Konten plus Kanäle ergaben jedes Mal exakt *S*. In einem Einzellauf lagen zum Beispiel 2292 auf den Konten und 2708 waren unterwegs, zusammen genau 5000. In drei direkt hintereinander ausgelösten Schnappschüssen war die Aufteilung 2095 + 2905, dann 972 + 4028 und schließlich 2463 + 2537, also jedes Mal 5000, obwohl zwischendurch über 80 % des gesamten Geldes gleichzeitig im Transit waren. Aufgefallen ist mir außerdem, dass derselbe Kanal mehrfach im Schnappschuss auftauchen kann: In meinem Lauf war der Kanal von Prozess 4 zu Prozess 2 mit drei Überweisungen gleichzeitig belegt (263, 16 und 70). Genau solche Fälle machen die Zähler-Lösung nötig. Mit mehreren, womöglich umsortierten Nachrichten auf einem Kanal muss die Aufzeichnung trotzdem stimmen.

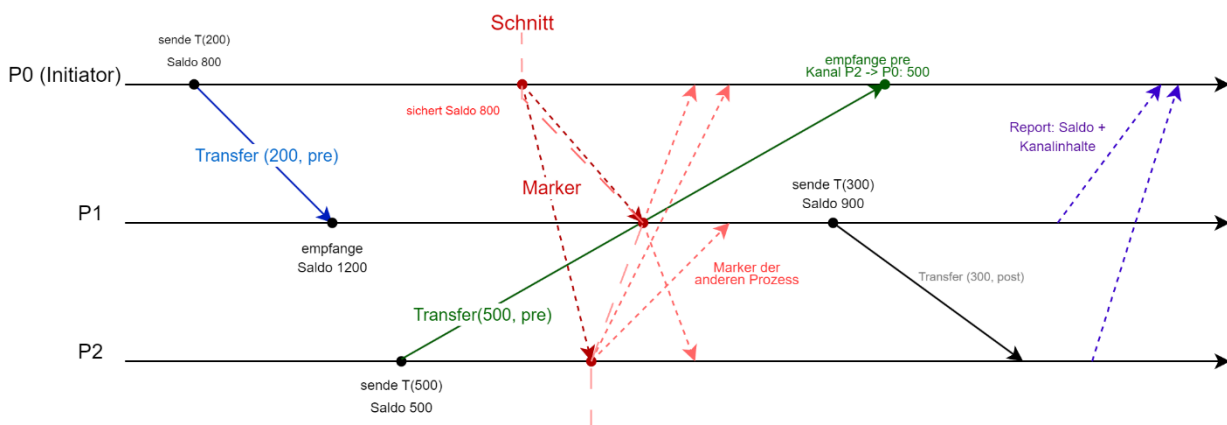


Abbildung 1: Konsistenter Schnitt mit Chandy-Lamport (Beispiel mit 3 Prozessen, $S = 3000$). Der Initiator *P0* sichert seinen Zustand und verschickt Marker; Transfer(500, *pre*) kreuzt den Schnitt und wird von *P0* als Kanalzustand aufgezeichnet.

Abbildung 1 zeigt den Ablauf an einem kleinen Beispiel mit drei Prozessen. Die gestrichelte Linie ist der Schnitt. Sie verläuft nicht senkrecht, weil die Marker die Prozesse zu unterschiedlichen Zeiten erreichen und jeder Prozess seinen Zustand erst beim ersten Marker sichert. Die Überweisung über 500 wurde vor dem Schnitt abgeschickt (*pre*) und erst danach empfangen. Sie kreuzt den Schnitt und landet in der Kanalaufzeichnung von *P0*. Zusammen mit den drei gesicherten Salden ergibt sich exakt *S*. Die *post*-Nachricht rechts gehört dagegen komplett in die Zeit nach dem Schnitt und wird korrekt ignoriert.

Zum Vergleich habe ich einen naiven Schnappschuss eingebaut: Der Initiator fragt nur die Kontostände ab und ignoriert die Kanäle komplett. Der war in meinen Läufen praktisch immer inkonsistent. Die drei naiven Schnappschüsse direkt vor den konsistenten ergaben 1066, 2488 und 961 statt 5000. Aus Sicht des Initiators „verschwinden“ also jedes Mal zwischen 2500 und 4000 Geldeinheiten, einfach weil die unterwegs befindlichen Überweisungen nirgends auftauchen.

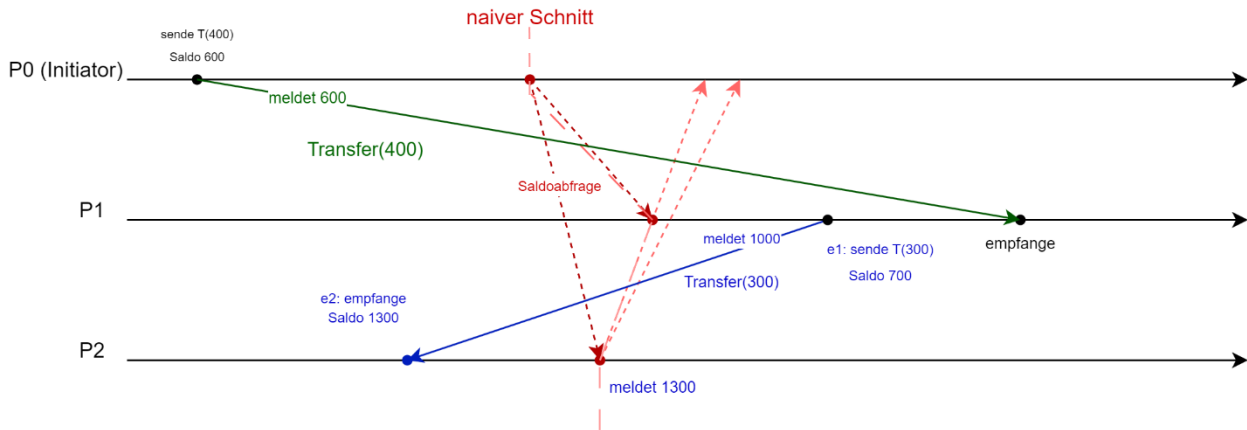


Abbildung 2: Inkonsistenter Schnitt beim naiven Schnappschuss. Transfer(400) ist unterwegs und wird nicht erfasst; bei Transfer(300) liegt der Empfang e_2 vor dem Schnitt, das Absenden e_1 aber dahinter (Nachricht aus der „Zukunft“).

Abbildung 2 zeigt die beiden Fehlerarten, die dabei zusammenkommen. Die Überweisung über 400 ist zum Zeitpunkt des Schnitts unterwegs: Der Sender hat sie schon abgebucht, der Empfänger noch nicht verbucht und weil niemand die Kanäle erfasst, fehlen diese 400 einfach. Grundsätzlicher ist der zweite Fall: P2 meldet seinen Saldo, nachdem er die Überweisung über 300 empfangen hat, P1 meldet seinen aber, bevor er sie abgeschickt hat. Der aufgezeichnete Zustand enthält damit den Empfang e_2 , nicht aber das zugehörige Absenden e_1 . Die 300 werden also doppelt gezählt. Genau das verletzt die Definition eines konsistenten Schnitts: Liegt ein Ereignis e_2 im Schnitt S und liegt e_1 kausal davor ($e_1 <_k e_2$), muss auch e_1 im Schnitt liegen. Der naive Schnappschuss kann das nicht garantieren, weil jeder Prozess zu einem unkoordinierten Zeitpunkt meldet. Im Beispiel ergibt sich $600 + 1000 + 1300 = 2900$, zusammengesetzt aus -400 (verschwundenes Geld) und $+300$ (doppelt gezähltes Geld). Meine Umsetzung verhindert beides: Das *pre*-Flag schließt Nachrichten aus der Zukunft aus, und die Kanalaufzeichnung fängt die unterwegs befindlichen Überweisungen ein.

Zum Schluss habe ich die Prozesszahl n variiert und die Kontrollnachrichten pro Schnappschuss gezählt: $n \cdot (n-1)$ Marker (jeder Prozess an jeden anderen) plus n Reports an den Initiator, in denen die aufgezeichneten Kanalinhalt gleich mitreisen. Die Tabelle zeigt die Werte sowie die durchschnittliche Zahl der Kanaleinträge (Überweisungspause 50–300 ms, Latenz 100–1000 ms; bei $n = 5$ aus meinen drei Schnappschüssen mit 23, 21 und 21 Einträgen):

n	Marker ($n \cdot (n-1)$)	Reports (n)	gesamt (n^2)	Kanaleinträge (ϕ)
3	6	3	9	≈ 12
5	20	5	25	≈ 22
10	90	10	100	≈ 43
20	380	20	400	≈ 93

Auffällig ist, dass die Zahl der Kontrollnachrichten mit n^2 wächst. Bei 20 Prozessen sind es schon 400 pro Schnappschuss. Das liegt daran, dass bei Chandy-Lamport jeder Prozess auf jedem seiner ausgehenden Kanäle einen Marker verschicken muss. In einem vollständig vernetzten System sind das zwangsläufig $n \cdot (n-1)$ Nachrichten. Dafür ist die Zahl unabhängig davon, wie viel die Anwendung gerade zu tun hat. Die Kanalinhalte reisen in den ohnehin nötigen Reports mit. Der Verkehr der Anwendung schlägt sich nur in der Größe der Reports nieder (bei $n = 20$ im Schnitt etwa 93 Kanaleinträge).

Damit lassen sich die in der Vorlesung genannten Nachteile von Schnappschuss-Algorithmen gut an den eigenen Zahlen festmachen. Das hohe Nachrichtenaufkommen ist offensichtlich: 400 zusätzliche Nachrichten pro Schnappschuss bei 20 Prozessen, quadratisch wachsend, was in einem großen System ein echtes Problem wäre. Und auch der Punkt „nicht verteilt genug“ zeigt sich: Zwar kann bei Chandy-Lamport jeder Prozess einen Schnappschuss initiieren, das Verfahren ist also dezentraler als eine Lösung mit festem Koordinator. Das Einsammeln der Teilzustände läuft aber trotzdem bei einem einzigen Prozess zusammen. Der Initiator ist somit Engpass und Single Point of Failure zugleich: Fällt er aus, sind alle bereits gesammelten Teilzustände wertlos.